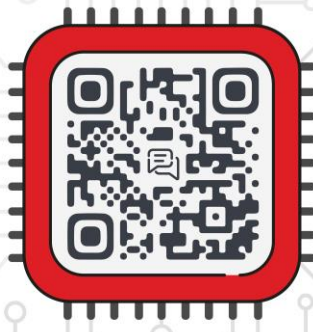




8



6



9

sp

TD

كلية الهندسة المعلوماتية

السنة الأولى

التجريد (Abstract)

د. ماهر صارم



RB Informatcs; 00/5/2026

محتوى مجاني غير مخصص للبيع التجاري

البرمجة 2

تعريف التجريد (Abstract):

هو كلاس ناقص، يحوي المعلومات الأساسية التي يجب أن تتواجد في الكلاسات التي سترث منه.

ينقسم التجريد إلى شيئين:

1- الكلاس التجريدي (Abstract Class):

هو كلاس ناقص يمثل فكرة عامة. لا يمكن إنشاء كائن (object) منه بشكل مباشر، وإنما يُنشأ ليكون أساساً (أباً) ترث منه كلاسات أخرى وتكمل بناءه.

مثال: كلاس "شكل هندسي"، لا يمكننا صنع شكل هندسي عشوائي، ولكن يمكننا إنشاء كلاس "مربع" يرث منه.

2- التابع التجريدي (Abstract Method):

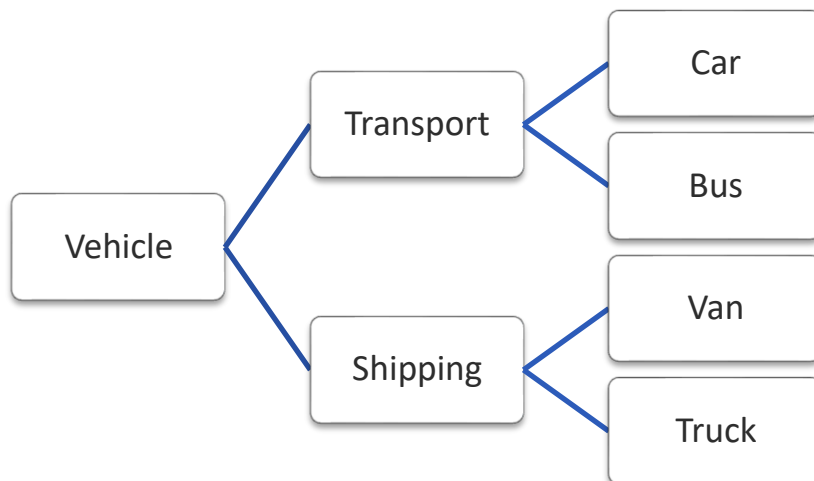
هي دالة تكتب داخل الكلاس التجريدي وتتكون من الإعلان عنها فقط (بدون كتابة أي كود أو أقواس مجمدة داخلها { })، وتنتهي بفاصلة منقوطة ;. وجودها يجبر أي كلاس ابن يرث من هذا الكلاس أن يقوم بإعادة كتابة هذه الدالة (@Override) ووضع الوظيفة الخاصة به داخلها.

مثال: دالة "حساب المساحة" لشكل هندسي مجهول لا يمكن كتابة كود لها في كلاس الأب، ولكن عند وراثة "المربع" يصبح مجبراً على كتابة كود حساب مساحة المربع.



مثال توضيحي لـ Abstract:

ليكن لدينا الخارطة التالية:



من الخارطة السابقة سيكون كلاس Vehicle كلاس تجريدي يحوي معلومات عامة يجب أن تتواجد في كل من كلاسي Transport و Shipping.

وكلا كلاسي Transport و Shipping تجريديين (Abstract) لأن كل منهما سيحوي على معلومات يجب أن تتواجد في الكلاسات التي سترث منها.

الكود:

```

public abstract class Vehicle {
    int num;
    Vehicle (int num){
        this.num=num;
    }
    abstract void printInfo();
}
  
```

```

public abstract class Shipping
extends Vehicle{
    int weight;
    Shipping(int num, int weight){
        super(num);
        this.weight=weight;
    }
}
  
```



```
public abstract class Transport extends Vehicle{
    int seatNumber;
    Transport(int num,int seatNumber){
        super(num);
        this.seatNumber=seatNumber;
    }
}
```

```
public class Truck extends Shipping{
    int length;
    Truck(int num,int weight,int length){
        super(num,weight);
        this.length=length;
    }
    @Override
    void printInfo(){
        System.out.println("Truck num is: "+num);
        System.out.println("Truck weight is: "+weight);
        System.out.println("Truck length is: "+length);
    }
}
```

```
public class Van extends Shipping{
    int capacity;
    Van(int num,int weight,int capacity){
        super(num,weight);
        this.capacity=capacity;
    }
    @Override
    void printInfo(){
        System.out.println("Van num is: "+num);
        System.out.println("Van weight is: "+weight);
        System.out.println("Van capacity is: "+capacity);
    }
}
```

إضافة _ تذكرة أريسييزية (ما معنى @Override) :

تستخدم @Override لإعلام المترجم (Compiler) بأن دالة الابن هي نسخة مُعدّلة ومخصصة، لتنفذ بدلاً من الدالة الأصلية الموروثة من كلاس الأب.

```
public class Car extends Transport{
    String type;
    Car(int num,int seatNumber,String type){
        super(num,seatNumber);
        this.type=type;
    }
    @Override
    void printInfo(){
        System.out.println("Car num is: "+num);
        System.out.println("Car seats number is:"+seatNumber);
        System.out.println("Car type: "+type);
    }
}
```

```
public class Bus extends Transport{
    int length;
    Bus(int num,int seatNumber,int length){
        super(num,seatNumber);
        this.length=length;
    }
    @Override
    void printInfo(){
        System.out.println("Bus num is: "+num);
        System.out.println("Bus seats number is:"+seatNumber);
        System.out.println("Bus length is: "+length);
    }
}
```

شرح الكود:

■ أولاً: كلاس (Vehicle):

نلاحظ أن الكلاس معرف كـ abstract وهذا لأنه كلاس عام ليجمع الصفة المشتركة لجميع المركبات وهي الرقم .num

ونلاحظ أيضاً التابع التجريدي printInfo ونرى أنه لا ينتهي بأقواس {} بل ينتهي بـ ؛ (فاصلة منقوطة) وذلك لأنه من النوع abstract، أي يجب على كل المركبات أن تحوي التابع printInfo الخاص بها.

■ ثانياً: كلاس (Shipping):

هذا الكلاس يمثل مركبات الشحن وهو يعتبر ابن الكلاس Vehicle ويرث منه الـ num وتابع printInfo، وهو أيضاً معرف كـ abstract لأنه مازال فكرة عامة (الشحن يندرج تحته Vang Truck)

ملاحظة:

لم نقوم بعمل Override لتابع printInfo في كلاس Shipping لأنه لا يمكن استدعائه بالأصل، وهو موجود عند الابن بالحالة العامة.

■ ثالثاً: كلاس Transport:

مثل كلاس Shipping تماماً لكنه يعبر عن المواصلات وبداخله متغير يعبر عن عدد المقاعد.

■ رابعاً: الكلاسات الأبناء

الكلاسات الفعلية وهي (Vang Truck) ترث من (Shipping) و (Car, Bus) ترث من (Transport)، نلاحظ أنه في هذه الكلاسات قمنا بعملية الـ Override لتابع printInfo ليعبر عن كل كلاس بطريقة مختلفة.

الخلاصة:

الهدف الأساسي من استخدام abstract في هذا المثال هو منع العشوائية وتنظيم الهيكلية. فنحن لا نريد من المبرمج أن يصنع كائناً هلامياً عاماً اسمه "مركبة"، بل نلجأ للتجريد لإجباره على تحديد نوعها الفعلي (هل هي شاحنة، باص، سيارة؟)، وفي نفس الوقت نضمن برمجياً أن جميع هذه المركبات تشترك في الخصائص الأساسية وتمتلك دالة لطباعة معلوماتها (printInfo).



إضافة أرييسزية (خلاصة فكرة الكود):

مستويات الكود الثلاثة (تدرج الوراثة):

◆ المستوى الأول: الأصل العام (Vehicle):

كلاس تجريدي (abstract class) يمثل فكرة "المركبة" بشكل عام.

◆ المستوى الثاني: التصنيف الفرعي (Shipping & Transport):

كلاسات تجريدية أيضاً، تقسم المركبات إلى نوعين رئيسيين:

مركبات شحن (Shipping): ترث الرقم وتضيف صفة (الوزن weight).

مركبات مواصلات (Transport): ترث الرقم وتضيف صفة (عدد المقاعد seatNumber).

◆ المستوى الثالث: المركبات الحقيقية (الكلاسات الفعلية):

هنا توجد المركبات الجاهزة للاستخدام والتي نحدد مواصفاتها بدقة، وتقوم بعمل @Override لتطبيق دالة الطباعة:

تحت الشحن: الشاحنة (Truck) وتضيف الطول، والفاوان (Van) وتضيف السعة.

تحت المواصلات: السيارة (Car) وتضيف النوع، والباص (Bus) وتضيف الطول.

انتهت المحاضرة الثامنة 🎉

